

Name : E. Rohan
Reg No : 192524482

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I E. Rohan - 192524482 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1.

Ans)

Scenario interpretation:

A school needs student, class, teacher, and grade information. A normalized design separates students, classes, teachers, and grades, then connects them through primary/foreign keys.

Relations:

STUDENT(StudentID PK, StudentName, ClassID FK, GradeID FK)

CLASS(ClassID PK, ClassName, TeacherID FK)

TEACHER(TeacherID PK, TeacherName)

GRADE(GradeID PK, GradeName)

Relationship:

STUDENT → CLASS → TEACHER

STUDENT → GRADE

Answer / SQL / Relational Algebra:

```
SELECT
  s.StudentID,
  s.StudentName,
  t.TeacherName AS ClassTeacher,
  g.GradeName
FROM Student s
JOIN Class c ON s.ClassID = c.ClassID
JOIN Teacher t ON c.TeacherID = t.TeacherID
JOIN Grade g ON s.GradeID = g.GradeID
ORDER BY s.StudentName;
```

Process / relationship diagram:

[Student → Class → Class Teacher; Student → Grade]

Key design rationale: Primary keys uniquely identify entities; foreign keys preserve relationships and reduce duplication. JOINS are used only where the scenario requires information from related relations.

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q2. Hospital Appointment Tracking

Ans)

A hospital must associate patients with doctors through appointments and retain the appointment date/time. The appointment relation is the key transaction entity because one patient and one doctor can have many appointments.

PATIENT(PatientID PK, PatientName, DOB, Phone)

DOCTOR(DoctorID PK, DoctorName, Specialization)

APPOINTMENT(AppointmentID PK, PatientID FK, DoctorID FK, AppointmentDate, AppointmentTime)

For “this month”, the filter should be based on the appointment date. GROUP BY counts appointments per doctor and HAVING keeps only doctors whose count exceeds 10.

Answer / SQL / Relational Algebra:

```
SELECT
    d.DoctorID,
    d.DoctorName,
    COUNT(a.AppointmentID) AS AppointmentCount
FROM Doctor d
JOIN Appointment a ON a.DoctorID = d.DoctorID
WHERE a.AppointmentDate >= DATE_TRUNC('month', CURRENT_DATE)
    AND a.AppointmentDate < DATE_TRUNC('month', CURRENT_DATE) + INTERVAL '1 month'
GROUP BY d.DoctorID, d.DoctorName
HAVING COUNT(a.AppointmentID) > 10;
```

Process / relationship diagram:

[Patient + Doctor → Appointment → Date filter → GROUP BY Doctor → HAVING count > 10]

Key design rationale: Primary keys uniquely identify entities; foreign keys preserve relationships and reduce duplication. JOINS are used only where the scenario requires information from related relations.

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q3. E-commerce: Products Never Ordered

Ans)

The important issue is that a product may exist in PRODUCT without a matching row in ORDER_ITEM. A correlated NOT EXISTS subquery is suitable because it checks each product for the absence of an order-line reference.

Answer / SQL / Relational Algebra:

```
SELECT
  p.ProductID,
  p.ProductName
FROM Product p
WHERE NOT EXISTS (
  SELECT 1
  FROM OrderItem oi
  WHERE oi.ProductID = p.ProductID
);
```

Process / relationship diagram:

[Product → Check matching OrderItem → Exists? No → Return product]

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q4. University: Students in Both DBMS and OS

Ans)

Assume:

STUDENT(SID, Name)

ENROLLMENT(SID, CID)

COURSE(CID, CourseName)

First select students enrolled in DBMS and students enrolled in OS. Their intersection gives students satisfying both conditions.

Answer / SQL / Relational Algebra:

```
DBMS_Students ←  $\pi_{SID} ($   
     $\sigma_{CourseName='DBMS'}$   
    (STUDENT  $\bowtie$  ENROLLMENT  $\bowtie$  COURSE)  
)
```

```
OS_Students ←  $\pi_{SID} ($   
     $\sigma_{CourseName='OS'}$   
    (STUDENT  $\bowtie$  ENROLLMENT  $\bowtie$  COURSE)  
)
```

```
Answer ← DBMS_Students  $\cap$  OS_Students
```

```
-- To display names:
```

```
 $\pi_{Name} (STUDENT \bowtie Answer)$ 
```

Process / relationship diagram:

[DBMS enrollment set $\cap$ OS enrollment set $\rightarrow$ Students enrolled in both]

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q5. Payroll: Below Department Average + 10% Update

Ans)

A department-level comparison requires a correlated subquery because the average salary must be calculated separately for each employee's department. The view identifies employees below that department average.

A practical design decision is to keep the view focused on identifying eligible employees and perform the update on the base table. This avoids trying to update an aggregate-derived view.

Answer / SQL / Relational Algebra:

```
CREATE VIEW Employees_Below_Dept_Avg AS
SELECT
  e.EmployeeID,
  e.EmployeeName,
  e.DepartmentID,
  e.Salary
FROM Employee e
WHERE e.Salary < (
  SELECT AVG(e2.Salary)
  FROM Employee e2
  WHERE e2.DepartmentID = e.DepartmentID
);
```

```
UPDATE Employee e
SET Salary = Salary * 1.10
WHERE e.EmployeeID IN (
  SELECT EmployeeID
  FROM Employees_Below_Dept_Avg
);
```

Process / relationship diagram:

[Employee → Department average → Salary below average? → View → 10% update on base table]

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Ans)

CUSTOMER(CustomerID PK, CustomerName)

TRANSACTION(TransactionID PK, AccountID FK, TransactionDate, Amount, TransactionType)

Answer / SQL / Relational Algebra:

Process / relationship diagram:

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q7. Retail: Daily Category Sales > Rs. 10,000

Ans)

The query must aggregate sales by category for the required day. SUM computes total sales, GROUP BY creates one result per category, and HAVING filters aggregated totals above Rs. 10,000. The date predicate is applied before grouping.

Answer / SQL / Relational Algebra:

```
SELECT
    p.Category,
    SUM(si.Quantity * si.UnitPrice) AS TotalSales
FROM Sale s
JOIN SaleItem si ON si.SaleID = s.SaleID
JOIN Product p ON p.ProductID = si.ProductID
WHERE s.SaleDate >= CURRENT_DATE
    AND s.SaleDate < CURRENT_DATE + INTERVAL '1 day'
GROUP BY p.Category
HAVING SUM(si.Quantity * si.UnitPrice) > 10000
ORDER BY TotalSales DESC;
```

Process / relationship diagram:

[Daily Sales → Product Category → GROUP BY Category → SUM → HAVING > Rs. 10,000]

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q8. Library: Members Who Borrowed From All Genres

Ans)

Assume:

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, GenreID)

GENRE(GenreID, GenreName)

BORROW(MemberID, BookID, BorrowDate)

Tuple relational calculus can express the “all genres” requirement using universal quantification. A member qualifies only if, for every genre, there exists a borrowed book belonging to that genre.

Answer / SQL / Relational Algebra:

```
{ m.MemberID, m.MemberName |  
  MEMBER(m) ∧  
  ∀g ( GENRE(g) →  
    ∃b ∃br (  
      BOOK(b) ∧  
      BORROW(br) ∧  
      br.MemberID = m.MemberID ∧  
      br.BookID = b.BookID ∧  
      b.GenreID = g.GenreID  
    )  
  )  
}
```

Process / relationship diagram:

[Member → For every Genre → Is there a borrowed Book in that Genre? → Yes for all → Return member]

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q9. Inventory: CHECK and FOREIGN KEY Enforcement

Ans)

A CHECK constraint enforces a valid domain rule on a column; a FOREIGN KEY enforces referential integrity between related tables. The example uses PRODUCT and SUPPLIER: stock cannot be negative, and every referenced supplier must exist.

Answer / SQL / Relational Algebra:

```
CREATE TABLE Supplier (  
    SupplierID INT PRIMARY KEY,  
    SupplierName VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100) NOT NULL,  
    StockQty INT NOT NULL CHECK (StockQty >= 0),  
    SupplierID INT NOT NULL,  
    CONSTRAINT fk_product_supplier  
        FOREIGN KEY (SupplierID)  
        REFERENCES Supplier(SupplierID)  
);
```

-- Valid:

```
INSERT INTO Supplier VALUES (1, 'ABC Supplies');  
INSERT INTO Product VALUES (101, 'Keyboard', 20, 1);
```

-- CHECK violation:

```
INSERT INTO Product VALUES (102, 'Mouse', -5, 1);
```

-- FOREIGN KEY violation:

```
INSERT INTO Product VALUES (103, 'Monitor', 10, 999);
```

Process / relationship diagram:

[INSERT/UPDATE → CHECK valid? → continue; invalid → rejected] → [INSERT/UPDATE → FK parent exists? → continue; missing parent → rejected]

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Q10. HR: Relational Schema + Materialized Views

Ans)

A compact HR schema separates employees, departments, attendance records, and performance reviews. Monthly summaries are good candidates for materialized views because summary calculations can be reused without recomputing the entire underlying data set for every report.

EMPLOYEE(EmployeeID PK, EmployeeName, DepartmentID FK, HireDate)

DEPARTMENT(DepartmentID PK, DepartmentName)

ATTENDANCE(AttendanceID PK, EmployeeID FK, AttendanceDate, Status)

PERFORMANCE(PerformanceID PK, EmployeeID FK, ReviewDate, Rating, Remarks)

Answer / SQL / Relational Algebra:

```
CREATE MATERIALIZED VIEW Monthly_Attendance_Summary AS
SELECT
    EmployeeID,
    DATE_TRUNC('month', AttendanceDate) AS MonthStart,
    COUNT(*) AS TotalDays,
    SUM(CASE WHEN Status = 'Present' THEN 1 ELSE 0 END) AS PresentDays,
    SUM(CASE WHEN Status = 'Absent' THEN 1 ELSE 0 END) AS AbsentDays
FROM Attendance
GROUP BY EmployeeID, DATE_TRUNC('month', AttendanceDate);
```

```
CREATE MATERIALIZED VIEW Monthly_Performance_Summary AS
SELECT
    EmployeeID,
    DATE_TRUNC('month', ReviewDate) AS MonthStart,
    AVG(Rating) AS AverageRating,
    COUNT(*) AS ReviewCount
FROM Performance
GROUP BY EmployeeID, DATE_TRUNC('month', ReviewDate);
```

```
-- Refresh when the DBMS-specific schedule/process requires it:
REFRESH MATERIALIZED VIEW Monthly_Attendance_Summary;
REFRESH MATERIALIZED VIEW Monthly_Performance_Summary;
```

Process / relationship diagram:

[Attendance records → Monthly aggregation → Materialized attendance view] → [Performance reviews → Monthly aggregation → Materialized performance view]

Key design rationale: Primary keys uniquely identify entities; foreign keys preserve relationships and reduce duplication. JOINS are used only where the scenario requires information from related relations.

Rubric alignment: The response identifies the scenario entities/issues, applies the requested DBMS concept, states the key decision/rationale, and presents the result in a structured form.

Submission Checklist

- All 10 questions are answered; each carries 5 marks.
- Diagrams / flow representations are included where they improve conceptual clarity.
- SQL and relational algebra are formatted for readability.
- Schema design identifies primary and foreign-key relationships where applicable.
- Answers emphasize scenario understanding, correct concept application, rationale, and clear structure.
- Student name, register number, and signature can be added before submission.

Student Name: _____

Register No.: _____

Signature: _____